Lab 2, Act 1 – Answer Key

From guess-and-check to systematic search

NRES 746 – Instructor Answer Key (not for student distribution)

This key follows the same section structure as the student handout. Questions with a single correct numeric or derived answer are answered directly; open-ended items (predictions, reflections, group discussion) are given sample answers/discussion points rather than a fixed "correct" response – use these to gauge whether a student's reasoning is on track, not as an exact-match rubric.

Part 1: Plan the search

1a. Propose a search range. Any range that comfortably brackets the previous week's guesses is reasonable – e.g. a from 0 to 3, b from 0 to 0.5. Students don't need to land on the "official" range used later in the handout ($a \in [0.1, 3]$, $b \in [0.02, 0.40]$); the point is practicing the habit of proposing a plausible range from prior knowledge before searching.

Combinations: for a range roughly this size, checking every 0.05 in a and every 0.01 in b gives on the order of a few thousand combinations. For the record, the official range used later gives an exact count:

```
avals <- seq(0.1, 3, by = 0.05)
bvals <- seq(0.02, 0.40, by = 0.01)
length(avals) * length(bvals)
```

```
## [1] 2301
```

Is a grid search still practical? Yes – 2,301 combinations is trivial for a computer (milliseconds), even though it would be absurd to do by hand.

1b. Design the algorithm. A correct plain-English answer should include all of the following steps:

1. Define a list (or range) of candidate a values and a list of candidate b values.
2. Set up storage for the SSR computed at every combination (e.g. a grid/matrix with one row per a value and one column per b value).
3. For every combination of a and b: compute the Ricker function's predicted values at the five tank sizes, compute the residuals (observed minus predicted), square them, and sum them to get the SSR for that combination. Store it.
4. After all combinations have been checked, find the smallest stored SSR and report the (a, b) combination that produced it.

Common gaps to watch for: forgetting that the *same* five data points get re-evaluated at every combination (not just once), or not being explicit about what gets stored (some students describe finding the best answer without describing how they'd search for it in the stored results).

Part 2: Implement the grid search

Actual implementation (either an explicit double loop or a vectorized `outer()` call will converge on the same answer):

```
ssr_grid <- outer(avals, bvals, Vectorize(function(a, b) sum((obs - ricker(xs, a, b))^2)))
best_idx <- which(ssr_grid == min(ssr_grid), arr.ind = TRUE)
```

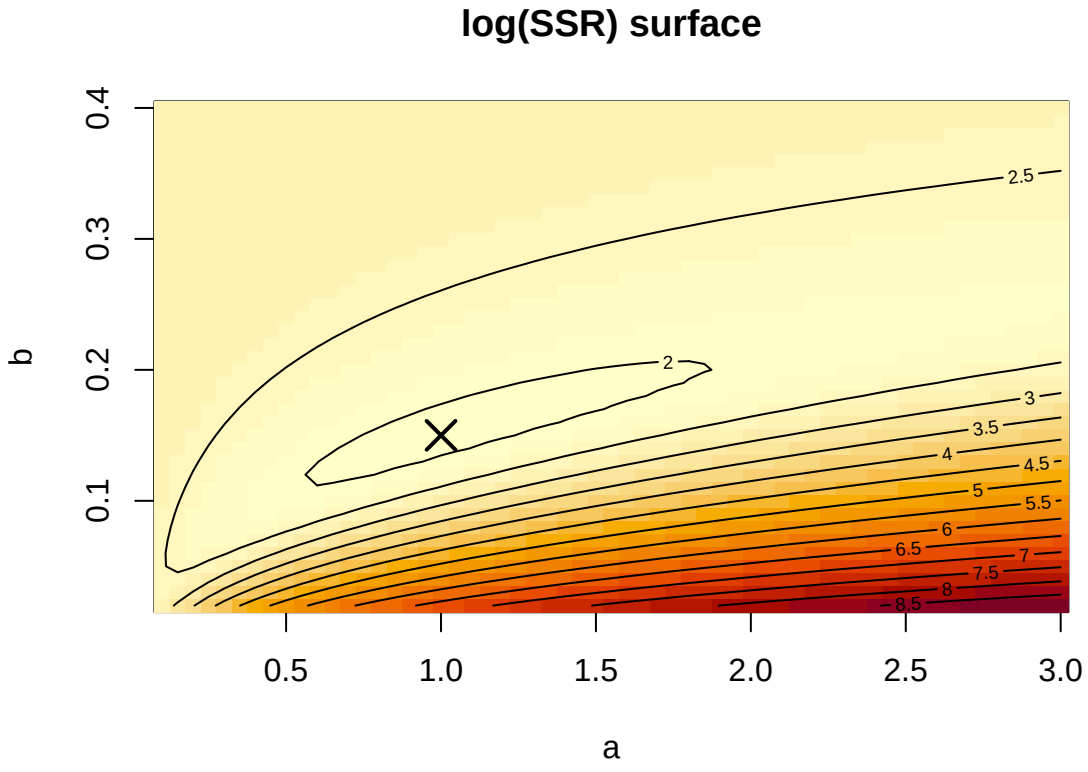
```r
a_best <- avals[best_idx[1, 1]]
b_best <- bvals[best_idx[1, 2]]
ssr_best <- ssr_grid[best_idx]
c(a = a_best, b = b_best, SSR = ssr_best)
```

```
##       a       b     SSR
## 1.00000 0.15000 6.73857
```

Results: best-fit $a = 1$, $b = 0.15$, SSR $= 6.74$. (If a student computed SSR against the 15 raw individual observations instead of the 5 tank-size means, they'll get the same (a, b) but a larger SSR, ≈ 24.9 – see the note on this in Act 2. Either is correct as long as they used it consistently.)

Heatmap/contour check. The minimum should sit comfortably inside the grid, not pinned to an edge. Plotting raw SSR is misleading here: it ranges from 6.74 at the best fit up to over 7,600 at the worst corner of the grid, so a linear color/contour scale crushes the entire good-fitting region into a single flat color and only shows structure in the worst corner – the "doesn't look good" symptom this plot has if you build it the naive way. Plotting `log(ssr_grid)` instead compresses that huge range and reveals the actual shape of the valley around the minimum:

```r
log_ssr <- log(ssr_grid)
image(avals, bvals, log_ssr, col = hcl.colors(30, "YlOrRd", rev = TRUE),
      xlab = "a", ylab = "b", main = "log(SSR) surface")
contour(avals, bvals, log_ssr, add = TRUE, nlevels = 12)
points(a_best, b_best, pch = 4, cex = 2, lwd = 2)
```

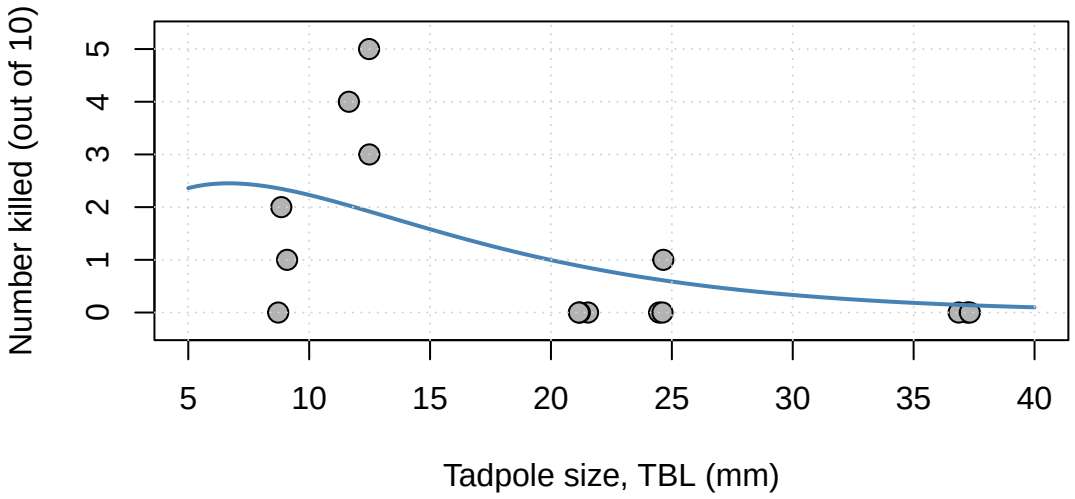


A closed contour loop should appear around the marked best fit – that closed shape *is* the valley: a student whose plot instead looks like solid color with contour lines bunched only in one corner has plotted raw SSR rather than log(SSR), which is worth flagging as a common pitfall, not a sign their grid search itself is wrong.

AI-assisted / self-coded reflection questions are open-ended by design – there's no fixed answer. Good responses typically note that the AI path was faster for getting *working syntax* but that the algorithmic thinking (Part 1) still had to happen first; weaker responses treat the AI as having done the whole task, which is a good prompt for follow-up discussion.

Does the best-fit curve peak where expected? No. Plotting it shows the answer clearly:

```
set.seed(1)
x_j <- frogs$TBL + runif(nrow(frogs), -0.6, 0.6)
plot(x_j, frogs$Kill, xlim = c(5, 40), ylim = c(-0.3, 5.3),
     xlab = "Tadpole size, TBL (mm)", ylab = "Number killed (out of 10)",
     pch = 21, bg = "grey70", cex = 1.4)
curve(ricker(x, a_best, b_best), from = 5, to = 40, lwd = 2, col = "steelblue", add = TRUE)
grid()
```



The curve declines from the left edge rather than rising to meet the spike at 12 mm. With only five aggregated data points, three of them at or near zero, minimizing SSR pulls the curve toward the many low points rather than chasing the one high one – not a bug, a real feature of fitting a rigid 2-parameter shape to sparse, spiky data. This sets up Act 2.

Group discussion. No fixed answer – the point is for students to reconcile any differences in (a, b) across paths (should be identical or nearly so, since grid search is deterministic) and flag their own muddiest points.